1. Ejercicios

1.1. Ejercicios de condicionales

1.- ¿Qué salida por pantalla se obtiene al ejecutarse el siguiente código?

```
double balance = 0;
balance = balance + 20; //Add quarter 1 profits (thousands).
balance = balance - 25; //Subtract quarter 1 expenses (thousands).
balance = balance + 30; //Add quarter 2 profits (thousands).
balance = balance - 25; //Subtract quarter 1 expenses (thousands).
if (balance < 0) {
    System.out.println("We're in the red!");
} else if (balance > 0) {
    System.out.println("We made a profit!");
} else {
    System.out.println("We broke even.");
}
```

(A) We're in the red!

(B) We made a profit!

(C) We broke even.

2.- ¿Qué salida por pantalla se obtiene al ejecutarse el siguiente código?

```
int dogs = 1;
int cats = 2;
if (dogs > 0 && cats == 0) {
    if (dogs > 1) {
        System.out.println("Dog lover");
    } else {
        System.out.println("Dog person");
    }
} else if (cats > 0 && dogs == 0) {
    if (cats > 1) {
        System.out.println("Cat lover");
    } else {
        System.out.println("Cat person");
    }
        System.out.println("Meow!");
} else if (cats > 0 && dogs > 0) {
    if (dogs > cats) {
        System.out.println("I guess you like dogs more");
```

```
        } else if (dogs == cats) {
            System.out.println("I guess you like both equally");
        } else {
            System.out.println("I guess you like cats more");
        }
    } else {
        System.out.println("What, don't you like pets?");
    }
```

3.- Supongamos que estamos desarrollando un videojuego en el que se pretende ganar a un enemigo. Para ello, tenemos objetos que pueden matarlo, pero, si somos vistos sin ningún objeto de ese tipo, nos matará.
En un trozo del código, tenemos dos variables booleanas (*sePuedeVerJugador* y *artículoMortal*).
sePuedeVerJugador es TRUE cuando el enemigo puede ver al jugador y FALSE en caso contrario.
articuloMortal es TRUE cuando el jugador tiene en su poder un artículo especial que hace posible poder derrotarlo.

Escribe las condiciones correctas para que se imprima "Ataque!" cuando el enemigo puede ver al jugador y el jugador puede derrotar al enemigo, "Huye!" cuando el enemigo puede ver al jugador pero el jugador no puede derrotar al enemigo y "Libre de peligro" cuando el jugador no está siendo visto por el enemigo.

4.- Supongamos que estamos desarrollando un código que nos indica si es un buen día para salir o no. En él, disponemos de dos variables de tipo boolean (*estaNevando* y *estaLloviendo*) y una variable de tipo float (*temperatura*). El programa deberá imprimir por pantalla "Quédate en casa" si está lloviendo, nevando o la temperatura es inferior a 5 ºC. En caso contrario, imprimirá "Vamos a salir".

5.- Supongamos que estamos desarrollando un despertador. Para ello, tenemos una variable de tipo entero llamada *diaSemana*. Ésta, puede tomar un valor entre 1 y 7 (ambos incluidos) que representarán los días de la semana, siendo 1 el lunes y 7 el domingo. También, tenemos una variable boolean llamada *esFestivo*, la cual será TRUE cuando el día sea festivo y FALSE en caso contrario.
Se pide que, en función del día de la semana, se imprima "Despertador conectado a las 7:00" de lunes a viernes y "Dormir!" los fines de semana y días festivos.

6.- Localiza el error que hay en el siguiente bloque de código:

```
double playbackPosition = 120;
boolean rewinding = true;
if (rewinding) {
    double rewindAmount = 0.1;
}
playbackPosition = playbackPosition - rewindAmount;
```

1.2. Ejercicios de bucles

1.- ¿Qué salida por pantalla se obtiene al ejecutarse el siguiente código?

```
int n = 10;
while (n < 50) {
    n = n * 2;
}
System.out.println(n);
```

(A) 10

(B) 40

(C) 50

(D) 80

2.- Desarrolla una función *factorial()*. Debe devolver el producto del número pasado por parámetro. Además, la función debe ser recursiva.

```
public int factorial(int n){
    // Completar
}
```

3.- Una cuenta de ahorro tiene un 5 % de interés anual. Se desea crear una función que permita determinar, mediante bucles, cuántos años hacen falta para tener en la cuenta 1.000.000 €. Esta función recibe un parámetro de entrada que será la cantidad monetaria depositada inicialmente en la cuenta y devuelve el número de años necesarios para tener 1.000.000 € o más en la cuenta.

```
public int tiempo(float balanceInicial){
    // Completar
}
```

4.- ¿Qué salida por pantalla se obtiene al ejecutarse el siguiente código?

```
int rows = 3;
for (int i = 1; i <= rows; i++) {
    String thisRow = "";
    for (int j = 0; j < i; j++) {
        thisRow = thisRow + "#";
```

```
        }
        System.out.println(thisRow);
    }
```

5.- Desarrolla una función que reciba por parámetro un número entero (n) y una letra y que devuelva un String que tenga n veces la letra dada.
Haz tres versiones distintas. En cada versión utiliza un tipo de bucle distinto.

6.- En el Monopoly, los jugadores deben lanzar dos dados para avanzar. Si los dos dados sacan el mismo número, se dice que "han salido dobles" y el jugador debe volver a tirar los dados. Si el jugador saca dobles tres veces, deberá ir directo a la cárcel. Se pide crear la función que tira los dados con todo lo necesario para realizar los siguientes pasos:

1. Generar dos números aleatorios entre 1 y 6 (ambos incluidos)
2. Si los números generados no son iguales, el método devolverá la suma de ambos.
3. Si los números generados son iguales, se volverá al paso 1.
4. Si han salido dobles tres veces, la función devolverá -1 para indicar que el jugador debe ir a la cárcel.

1.3. Ejercicios de métodos

1.- Se propone el siguiente código y las siguientes cuestiones a realizar:

```
pubic class Ejercicio3{

    public static void main (String[] args){
        int a = 24;
        int b = 5;
        // Completar el método
    }
}
```

Cuestiones:

1. Crear un método que reciba por parámetro los dos números y devuelva la suma de ambos.
2. Crear un método que reciba por parámetro los dos números y devuelva la resta de ambos.
3. Crear un método que reciba por parámetro los dos números y devuelva la multiplicación de ambos.
4. Crear un método que reciba por parámetro los dos números y devuelva la división de ambos.
5. Crear un método que reciba por parámetro los dos números y devuelva el módulo de ambos.
6. Crear un método que reciba por parámetro los dos números e imprima por pantalla cuál es el mayor de los dos y cuál el menor.
7. Completar el método `main()` para invocar todos los métodos y que imprima por pantalla los resultados de las cuestiones 1, 2, 3, 4 y 5.

2.- Corren los años 80, un poderoso banco está intentando dar el gran salto a los nuevos ordenadores y quiere crear un programa para que sus clientes utilicen sus servicios desde casa (lo que hoy en día es la banca online). Para ello, van a crear un programa en línea de comandos.

Para ello, se quiere que se muestre el siguiente menú atendiendo a las condiciones:

```
***** BIENVENIDO A SU BANCO *****
Seleccione una de las siguientes opciones en el teclado numérico:
    1.- Consultar saldo
    2.- Realizar una transferencia
    3.- Consultar/cambiar los datos personales
    4.- Solicitar un préstamo
    5.- Pagar impuestos
    0.- Cerrar sesión
********************************
```

Condiciones:

- El menú se repetirá hasta que el usuario cierre sesión (es decir, que marque 0).
- Comprobar que el usuario ha dado un número que está dentro de los parámetros (1,2,3,4,5 ó 0).
- Si el usuario da un valor que no está entre los del menú, imprimir el siguiente mensaje: **"Ha introducido un valor que no está en el menú"**.
- Cada una de las opciones (excepto la opción 0) redirigen a un método distinto. En este caso no hace falta definirlos, por lo que se puede poner que redirigen a `metodo1(), metodo2()` y así sucesivamente.
- Cuando el usuario cierre sesión (opción 0), se deberá imprimir el siguiente mensaje antes de cerrar el programa: **"Muchas gracias por utilizar nuestros servicios. ¡Hasta pronto!"**.

1.4. Ejercicios de Arrays y Matrices

1.- Se propone el siguiente código y las siguientes cuestiones a realizar:

```
import java.util.ArrayList;

public class Ejercicio1{

    static String[] arrayDeAlimentos;

    public static void main(String[] args){
        introducirDatos();
        mostrarDatos();
        System.out.println(clonarArray());
        clonarArrayV2();
        System.out.println(agregarArrayList());
    }

    public static void introducirDatos(){
        // Completar el método
    }

    public static void mostrarDatos(){
        // Completar el método
    }

    public static String clonarArray(){
        // Completar el método
        return "";
    }

    public static void clonarArrayV2(){
        // Completar el método
    }

    public static String agregarArrayList(){
        // Completar el método
        return "";
    }
}
```

Cuestiones:

1. Completar el método `introducirDatos()` para que dentro del array se encuentren los siguientes elementos: “Hamburguesa”, “Perrito caliente”, “Croquetas”, “Sopa castellana”, “Crema de calabacín”.

2. Completar el método `mostrarDatos()` para que se muestren los datos que hay dentro del arrayDeAlimentos.

3. Completar el método `clonarArray()` en el que se debe crear un nuevo array (llamado arrayClonado) que contenga los mismos elementos que el arrayDeAlimentos. Se debe hacer con el método `.clone()`.
   Una vez clonado, devolver el array en forma de String (se imprimirá por pantalla).

4. Completar el método `clonarArrayV2()` en el que se debe crear un nuevo array (llamado arrayClonadoV2) que contenga los mismos elementos que el arrayDeAlimentos. En este caso, no se podrá utilizar el método `.clone()`.

5. Completar el método `agregarArrayList()` en donde crearemos un ArrayList y añadiremos a él los elementos que tenemos del arrayDeAlimentos, Una vez hecho esto, crearemos una variable de tipo String en la que guardaremos los elementos del ArrayList separados por comas ','. Finalmente, devolveremos ese String (para que se imprima por pantalla).

2.- Se propone el siguiente código y las siguientes cuestiones a realizar:

```
public class Ejercicio2{
    static String[] nombres1 = {"Pilar", "Pedro", "Juan", "Teresa",
        "Leticia", "Nicolás", "Manuel", "Carolina"};
    static String[] nombres2 = {"Leticia", "Juan Pablo", "Virginia",
        "Manuel", "Mario", "María", "Ana", "Nicolás"};

    public static void main(String[] args){
        comprobarNombres();
        todosLosNombres();
    }

    public static void comprobarNombres(){
        // Completar el método
    }

    public static void todosLosNombres(){
        // Completar el método
    }
}
```

Cuestiones:

1. Completar el método `comprobarNombres()` en el cual se deberá comparar el array `nombres1` con el array `nombres2` e imprimir por pantalla los siguientes mensajes:

   - Si el nombre está almacenado en los dos arrays:
     <nombre>está en los dos arrays.
     Leticia está en los dos arrays.

- Si el nombre **no** está almacenado en los dos arrays:
  <nombre>no está en los dos arrays.
  Pilar no está en los dos arrays.

2. Completar el método `todosLosNombres()` donde se deberá crear un array que contenga todos los nombres (los del array `nombres1` y los del array `nombres2`) sin repetir ninguno. Una vez creado, imprimirlo por pantalla.

3.- Un grupo de amigos quieren jugar al juego de los barcos, pero lo único que tienen a mano es un ordenador, y tú los vas a ayudar.

Para facilitar las cosas, los barcos ocuparán una única casilla, y se deberán poner 7 barcos.

Las casillas donde estarán los barcos estarán definidas por una matriz de dimensiones X (es decir, nosotros no lo sabemos).

En la matriz, las posiciones donde existen barcos tendrán un **1** y donde hay agua tendrán un **0**.

Imaginemos una matriz como la siguiente:

	0	**1**	**2**	**3**	**4**	**5**	**6**
0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0
2	0	0	0	0	1	0	0
3	0	1	0	0	0	0	0
4	0	0	0	0	1	0	0
5	0	0	1	0	0	0	0
6	1	0	0	0	0	0	0

Se pide crear un método que reciba por parámetro la matriz del jugador, una *coordenada X* (que corresponderá con las filas) y una *coordenada Y* (que corresponderá con las columnas). Dicho método deberá devolver los siguientes resultados:

- Si se ha dado a un barco: **Tocado y hundido**.
- Si no se ha dado a un barco: **Agua**.
- Si se ha dado a un barco y ya no quedan más barcos a flote: **Tocado, hundido y fin de la partida**.

Por ejemplo, si el usuario da las siguientes coordenadas → (4,5), el programa deberá responder **"Tocado y hundido"**. Si se pasan las siguientes coordenadas → (0,0), el programa responderá **"Agua"**.

4.- Los precios de una autopista vienen determinados por la suma del precio de salida y del precio de llegada.

En la siguiente tabla se observan los precios que hay. Los números que hay encima y en el lateral izquierdo indican las posiciones de salida y llegada (por ejemplo, la autopista puede ser la AP-61 de Segovia a Madrid y las diferentes salidas son los municipios que hay por el camino, como Villacastín,

	0	**1**	**2**	**3**	**4**	**5**	**6**
0		5	10	15	20	25	30
1	5		15	20	25	30	35
2	10	15		25	30	35	40
3	15	20	25		35	40	45
4	20	25	30	35		45	50
5	25	30	35	40	45		55
6	30	35	40	45	50	55	

Cuadro 1: Tabla de precios

San Rafael, El Escorial...). La salida a la autopista se representa con la posición vertical y la llegada se encuentra en la horizontal.

Como podrás observar, la diagonal es gris, lo que significa que ese trayecto no es realizable ya que saldrías y llegarías al mismo sitio.

En vista de los datos ofrecidos, se pide crear un programa que realice lo siguiente:

1. Construir la *Tabla de precios*.
2. Reciba por teclado una posición de inicio y otra de fin e imprima por pantalla el siguiente mensaje → **"El precio del trayecto <inicio>- <fin>es de <precio> €"**.
   Por ejemplo, para el trayecto 2-6 imprimirá → "El precio del trayecto 2 - 6 es de 40 €"
   Hay que tener en cuenta que si un trayecto no se puede realizar (por ejemplo, el trayecto 0-0 o el trayecto 5-5), el programa deberá imprimir el siguiente mensaje → **"El trayecto <inicio>-<fin>no es realizable"**.
   Por ejemplo, para el trayecto 5-5 se imprimirá → "El trayecto 5-5 no es realizable"

A tener en cuenta:

- Se pueden realizar tantos métodos como se crea oportuno.
- Para simplificar las cosas, en los trayectos que no sean realizables, en la casilla se pondrá *-1*.
- En este caso, la tabla es de *6x6*, pero en cualquier otro caso puede tener otro tamaño. Eso sí, siempre será una matriz cuadrada.
- El usuario no sabe cuánto mide la tabla, por lo que podría introducir un dato mayor a la dimensión y producirse una excepción que habrá que controlar.